

03-DataVisualization

September 4, 2025

1 Feature Manipulation in Pandas

Here let's look at a different dataset that will allow us to really dive into some meaningful visualizations. This data set is publically available, but it is also part of a Kaggle competition.

You can get the data from here: <https://www.kaggle.com/c/titanic-gettingStarted> or you can use the code below to load the data from GitHub.

There are lots of iPython notebooks for looking at the Titanic data. Check them out and see if you like any better than this one!

When going through visualization options, I recommend the following steps: - Would you like the visual to be interactive? - Yes, Does it have a lot of data? - No, Use plotly or bokeh - Yes, sub-sample and then use plotly/bokeh - Yes, think about using Turi for large data - No, Does seaborn have a built-in function for plotting? - Yes, use seaborn - No, Does Pandas support the visual? - Yes, use pandas - No, use low level matplotlib

Look at various high level plotting libraries like: - Altair (<https://altair-viz.github.io>) - Bokeh (<http://bokeh.pydata.org/en/latest/>) - And many others...

1.1 Adding Dependencies (for Jupyter Lab)

- `conda install -c conda-forge missingno`
- `conda install nodejs`
- `jupyter labextension install @jupyterlab/plotly-extension`

2 Loading the Titanic Data for Example Visualizations

```
[1]: # load the Titanic dataset
import pandas as pd
import numpy as np

print('Pandas:', pd.__version__)
print('Numpy:', np.__version__)

df = pd.read_csv('https://raw.githubusercontent.com/eclarson/
↳DataMiningNotebooks/master/data/titanic.csv') # read in the csv file

df.head()
```

Pandas: 2.3.2

Numpy: 2.2.5

```
[1]: PassengerId  Survived  Pclass  \
0           1         0         3
1           2         1         1
2           3         1         3
3           4         1         1
4           5         0         3
```

```

                                Name      Sex  Age  SibSp  \
0                Braund, Mr. Owen Harris   male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th... female  38.0      1
2                Heikkinen, Miss. Laina   female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)   female  35.0      1
4                Allen, Mr. William Henry   male  35.0      0
```

```

      Parch      Ticket    Fare Cabin Embarked
0         0   A/5 21171    7.2500   NaN        S
1         0    PC 17599   71.2833   C85        C
2         0 STON/O2. 3101282    7.9250   NaN        S
3         0    113803   53.1000  C123        S
4         0    373450    8.0500   NaN        S
```

```
[2]: # note that the describe function defaults to using only some variables
df.describe()
```

```
[2]:
```

	PassengerId	Survived	Pclass	Age	SibSp	\
count	891.000000	891.000000	891.000000	714.000000	891.000000	
mean	446.000000	0.383838	2.308642	29.699118	0.523008	
std	257.353842	0.486592	0.836071	14.526497	1.102743	
min	1.000000	0.000000	1.000000	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	38.000000	1.000000	
max	891.000000	1.000000	3.000000	80.000000	8.000000	

```

      Parch      Fare
count  891.000000  891.000000
mean    0.381594   32.204208
std     0.806057   49.693429
min     0.000000    0.000000
25%     0.000000    7.910400
50%     0.000000   14.454200
75%     0.000000   31.000000
max     6.000000  512.329200
```

```
[3]: print(df.dtypes)
      print('=====')
      print(df.info())
```

```

PassengerId      int64
Survived          int64
Pclass           int64
Name             object
Sex              object
Age             float64
SibSp            int64
Parch            int64
Ticket           object
Fare            float64
Cabin           object
Embarked         object
dtype: object
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   PassengerId     891 non-null   int64
 1   Survived        891 non-null   int64
 2   Pclass          891 non-null   int64
 3   Name            891 non-null   object
 4   Sex             891 non-null   object
 5   Age            714 non-null   float64
 6   SibSp           891 non-null   int64
 7   Parch           891 non-null   int64
 8   Ticket          891 non-null   object
 9   Fare            891 non-null   float64
10   Cabin           204 non-null   object
11   Embarked        889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

2.1 Questions we might want to ask:

- What percentage of passengers survived the Titanic disaster?
- What percentage survived in each class (first, coach, etc.)?
- How many people traveled in each class? How many classes are there?

```
[4]: # the percentage of individuals that survived on the titanic
      num_survived = sum(df.Survived == 1)
      print(num_survived, "survivors")
```

```
print(num_survived/len(df)*100.0, "%")
```

```
342 survivors
38.38383838383838 %
```

2.2 Grouping the Data

```
[5]: # Lets aggregate by class and count survival rates
df_grouped = df.groupby(by='Pclass')

for val,grp in df_grouped:
    print(f'There were {len(grp)} people traveling in {val} class.')
```

```
There were 216 people traveling in 1 class.
There were 184 people traveling in 2 class.
There were 491 people traveling in 3 class.
```

```
[6]: # an example of using the groupby function with a data column
print(df_grouped['Survived'].sum())
print('-----')
print(df_grouped.Survived.count())
print('-----')
print(df_grouped.Survived.sum() / df_grouped.Survived.count())

# might there be a better way of displaying this data?
```

```
Pclass
1    136
2     87
3    119
Name: Survived, dtype: int64
-----

Pclass
1    216
2    184
3    491
Name: Survived, dtype: int64
-----

Pclass
1    0.629630
2    0.472826
3    0.242363
Name: Survived, dtype: float64
```

3 Cleaning the Dataset

Let's start by visualizing some of the missing data in this dataset. We will use the `missingno` package to help visualize where the data contains NaNs. This is a great tool for looking at nan values and how we might go about filling in the values.

For this visualization, we can use a visualization library called `missingno` that has many types of visuals for looking at missing data in a dataframe. I particularly like the `matrix` visualization, but there are many more to explore: - <https://github.com/ResidentMario/missingno>

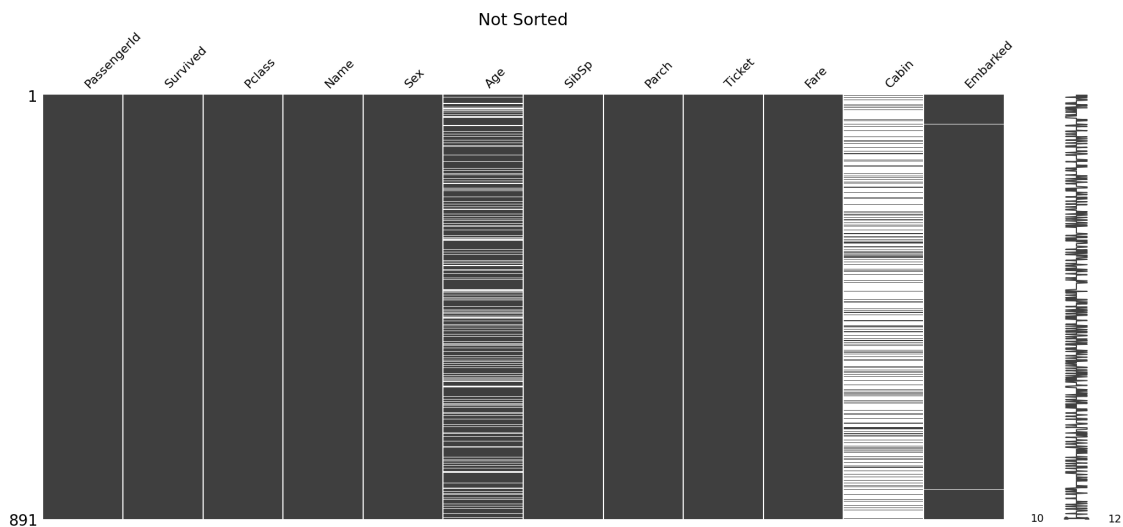
3.0.1 Plot Type One: Filter Bar

```
[7]: # this python magics will allow plot to be embedded into the notebook
import matplotlib
import matplotlib.pyplot as plt
import warnings
warnings.simplefilter('ignore', DeprecationWarning)
%matplotlib inline

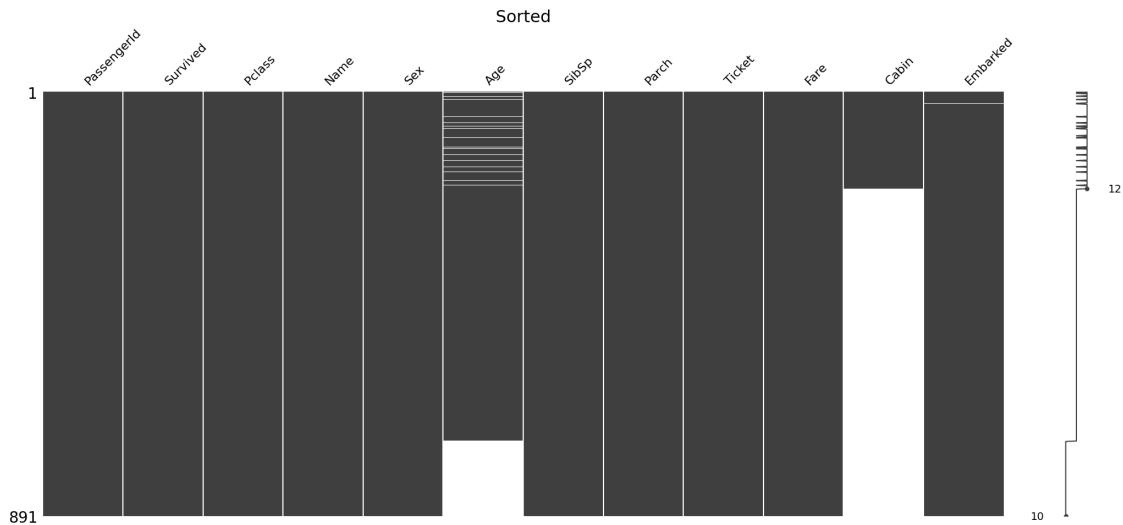
# External package: conda install missingno
import missingno as mn

mn.matrix(df)
plt.title("Not Sorted", fontsize=22)

plt.figure()
mn.matrix(df.sort_values(by=["Cabin", "Age"]))
plt.title("Sorted", fontsize=22)
plt.show()
```



<Figure size 640x480 with 0 Axes>



Let's clean the dataset a little before moving on

1. Remove attributes that just aren't useful for us
 - PassengerID is simply an identifier and not useful for analysis
 - Ticket is simply an identifier and not useful for analysis
 - Cabin is largely not present so not useful for analysis
 - Name would be very difficult to analyze without prior processing

```
[8]: df.drop(columns=['PassengerId', 'Name', 'Cabin', 'Ticket'], inplace=True)

"""
# Create a method to determine a unique column name
def unique_column(df, name_attempt):
    actual_name = name_attempt
    counter = 2
    while actual_name in df.columns:
        actual_name = "%s_%d" % (name_attempt, counter)
        counter += 1
    return actual_name

# Create a method to automatically one hot-code categorical data
def onehotcoder(df, column):
    col_uniques = df[column].unique()
    sorted_uniques = sorted(col_uniques[~pd.isnull(col_uniques)]) # Remove NaN_
    ↪values
    for category in sorted_uniques:
        category_name = unique_column(df, "%s_%s" % (column, category))
        df[category_name] = np.where(df[column] == category, True, False)
    del df[column]
"""
```

```
def onehotcoder(df, *args):
    cols = list(args)
    return pd.get_dummies(df, prefix = cols, columns = cols, dummy_na = False)

df = onehotcoder(df, 'Sex', 'Embarked')

df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Survived        891 non-null   int64
1   Pclass          891 non-null   int64
2   Age             714 non-null   float64
3   SibSp           891 non-null   int64
4   Parch           891 non-null   int64
5   Fare            891 non-null   float64
6   Sex_female      891 non-null   bool
7   Sex_male        891 non-null   bool
8   Embarked_C      891 non-null   bool
9   Embarked_Q      891 non-null   bool
10  Embarked_S      891 non-null   bool
dtypes: bool(5), float64(2), int64(4)
memory usage: 46.2 KB
```

```
[8]:
```

	Survived	Pclass	Age	SibSp	Parch	Fare	Sex_female	Sex_male	\
0	0	3	22.0	1	0	7.2500	False	True	
1	1	1	38.0	1	0	71.2833	True	False	
2	1	3	26.0	0	0	7.9250	True	False	
3	1	1	35.0	1	0	53.1000	True	False	
4	0	3	35.0	0	0	8.0500	False	True	

	Embarked_C	Embarked_Q	Embarked_S
0	False	False	True
1	True	False	False
2	False	False	True
3	False	False	True
4	False	False	True